Humanoid Robot Modeling and Dynamics

Step-by-step implementation guide — Euler-angle floating base

Rewritten from the technical study notes. Same model and equations, reorganized into steps you can code and test one at a time. All numbers refer to `biped_simple_v1` (pelvis bar + two 5-joint legs, 24.004 kg) and the code in `robot_model.py`.

How to use this guide

Each step has the same four parts. **Goal**: what the step gives you. **Math**: only the equations needed for that step. **Implement**: the function to write and a code sketch. **Test**: a check with a known answer. Do not move to the next step until the test passes; every later step depends on the earlier ones. Steps 1–4 are already implemented and tested in `robot_model.py`.

Step	Topic	You get	Status
0	Conventions	Coordinates, frames, symbols	—
1	Kinematic tree	Links, joints, masses from the URDF	done
2	Forward kinematics	World pose of every link	done
3	Base orientation	$R(\boldsymbol{\theta}_b)$, $E(\boldsymbol{\theta}_b)$, $\dot{E}$	done ($\dot{E}$ new)
4	Jacobians	J_{v_i}, J_{ω_i}, contact J_c	done
5	Mass matrix	$M(\mathbf{q})$	to do
6	Gravity	$G(\mathbf{q})$	to do
7	Coriolis/centrifugal	$C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$	to do
8	Free-flight simulation	$\ddot{\mathbf{q}}$ and time stepping	to do
9	Ground contact (penalty)	Robot stands on the floor	to do
10	Rigid contact (optional)	Exact constrained dynamics	later
11	First controller	Standing with joint PD	later

Step 0 | Conventions and notation

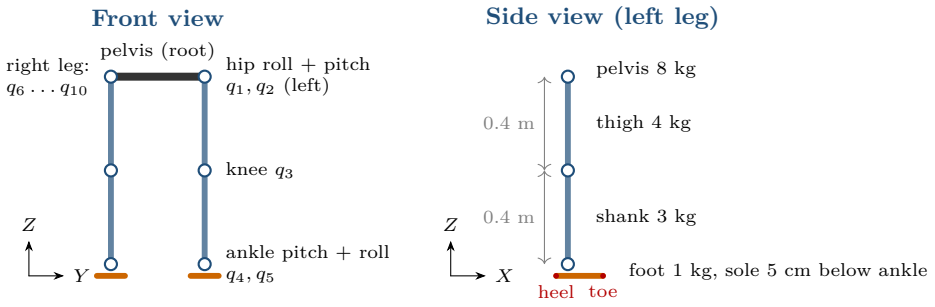
World frame: X forward, Y left, Z up. Units m, kg, rad, s. Gravity $-g\,\mathbf{e}_z$ with $g = 9.81$.

Generalized coordinates ($n = 10$ joints, so 16 numbers):

$$\mathbf{q} = \big[\underbrace{x,\ y,\ z}_{\mathbf{p}_b},\ \underbrace{\phi,\ \theta,\ \psi}_{\boldsymbol{\theta}_b\ \text{(roll, pitch, yaw)}},\ \underbrace{q_1, \ldots, q_{10}}_{\mathbf{q}_j}\big]^{\mathsf{T}} \in \mathbb{R}^{16}, \qquad \dot{\mathbf{q}} \in \mathbb{R}^{16}. \tag{1}$$

Because Euler angles are three ordinary coordinates, $\dot{\mathbf{q}}$ is simply the time derivative of $\mathbf{q}$ (code: `v = qdot`). The Euler-angle rates $\dot{\boldsymbol{\theta}}_b$ are *not* the angular velocity; Step 3 gives the conversion.

Euler convention (URDF / ROS): $R(\boldsymbol{\theta}_b) = R_z(\psi)\, R_y(\theta)\, R_x(\phi)$. Always use this one; other orderings give a different $E(\boldsymbol{\theta}_b)$.



Symbols used in every step

Symbol	Meaning	Code
$R_i,\ \mathbf{o}_i$	world rotation and origin of link i's frame	`frames[link]`
$\mathbf{c}_i$	center of mass of link i in its own frame	`link.com`
$\mathbf{p}_i = \mathbf{o}_i + R_i\mathbf{c}_i$	world position of link i's center of mass	
$m_i,\ I_i^{\text{body}}$	mass; inertia about the CoM in link axes	`link.mass`, `link.inertia`
$I_i^W = R_i I_i^{\text{body}} R_i^{\mathsf{T}}$	inertia about the CoM in world axes	
$\mathbf{a}_j,\ \mathbf{p}_j$	world axis and world position of joint j	from Step 2
$\mathbf{v}_i,\ \boldsymbol{\omega}_i$	CoM linear velocity, angular velocity of link i	
$[\mathbf{a}]_\times$	cross-product matrix: $[\mathbf{a}]_\times\mathbf{b} = \mathbf{a}\times\mathbf{b}$	`skew(a)`
$S = [\,0_{10\times 6}\ \ I_{10}\,]$	selects the actuated joints	

Step 1 | Kinematic tree

[done: `RobotModel.__init__`]

Goal. Read the URDF into a tree of links and joints, rooted at the pelvis.

Store, per link: name, m_i, $\mathbf{c}_i$, I_i^{body}. **Per joint:** type, parent, child, origin offset $(\mathbf{d}_j, R_{o,j})$, local axis, limits, and the index of its angle in $\mathbf{q}$.

Implement. (1) Parse `<link>` and `<joint>`. (2) The root is the link that is never a child. (3) Order joints parent-before-child (breadth-first from the root). (4) Number the actuated joints in file order: joint k is `q[6+k]`. (5) For each link, list the actuated joints on its path to the root (its *ancestor joints*); Step 4 needs this list.

Test — done when:
Root is `pelvis`; 10 actuated joints; $n_q = 16$; total mass = 24.004 kg; `left_hip_roll_joint` is `q[6]`, `right_ankle_roll_joint` is `q[15]`.

Step 2 | Forward kinematics

[done: `forward_kinematics`]

Goal. World rotation R_i and origin $\mathbf{o}_i$ of every link for a given $\mathbf{q}$.

Math. Start at the pelvis, $R_b = R(\boldsymbol{\theta}_b)$, $\mathbf{o}_b = \mathbf{p}_b$. For each joint, parents first, apply the fixed mounting offset and then the joint motion:

$$R_{\text{joint}} = R_{\text{parent}}\,R_{o,j}, \qquad \mathbf{o}_{\text{joint}} = \mathbf{o}_{\text{parent}} + R_{\text{parent}}\,\mathbf{d}_j \qquad \text{(URDF <origin>)} \tag{2}$$

$$R_{\text{child}} = R_{\text{joint}}\,\mathrm{Rot}(\hat{\mathbf{a}}_j, q_j), \qquad \mathbf{o}_{\text{child}} = \mathbf{o}_{\text{joint}} \qquad \text{(URDF <axis>, angle } q_j\text{)} \tag{3}$$

with the Rodrigues formula for a rotation by angle q about a unit axis $\hat{\mathbf{a}}$ in the joint frame:

$$\mathrm{Rot}(\hat{\mathbf{a}}, q) = I + \sin q\,[\hat{\mathbf{a}}]_\times + (1-\cos q)\,[\hat{\mathbf{a}}]_\times^2. \tag{4}$$

This is the notes' ${}^{W}T_i = {}^{W}T_{\text{parent}(i)}\,{}^{\text{parent}(i)}T_i(q_i)$, split into its fixed and moving parts. The joint's world axis and position, needed in Step 4, are $\mathbf{a}_j = R_{\text{child}}\hat{\mathbf{a}}_j$ and $\mathbf{p}_j = \mathbf{o}_{\text{child}}$.

Center of mass of the whole robot: $\mathbf{p}_{\text{CoM}} = \frac{1}{m}\sum_i m_i\,\mathbf{p}_i$, $m = \sum_i m_i$.

Test — done when:
Zero pose with $z = 0.85$: hips at $(0, \pm 0.1, 0.85)$, knees at $z = 0.45$, ankles at $z = 0.05$, heel $(-0.05, 0.1, 0)$, toe $(0.15, 0.1, 0)$, CoM $(0.0042,\ 0,\ 0.5625)$. Check by hand: $(8\cdot 0.85 + 8\cdot 0.65 + 6\cdot 0.25 + 2\cdot 0)/24.004 = 0.5625$. Plot a few poses and confirm the knees bend forward (`step1_fk_plot.py`).

Step 3 | Base orientation: $R(\boldsymbol{\theta}_b)$, $E(\boldsymbol{\theta}_b)$ and $\dot{E}$

[done: `rpy_to_R`, `euler_rate_matrix`]

Goal. Convert pelvis Euler angles to a rotation matrix, and Euler-angle *rates* to the physical angular velocity.

Rotation. With $c_\bullet = \cos$, $s_\bullet = \sin$:

$$R(\boldsymbol{\theta}_b) = R_z(\psi)R_y(\theta)R_x(\phi), \quad R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi & c_\phi \end{bmatrix}, \; R_y = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}, \; R_z = \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{5}$$

Angular velocity. Each Euler rate turns the body about its own current axis: $\dot{\phi}$ about $R_zR_y\mathbf{e}_x$, $\dot{\theta}$ about $R_z\mathbf{e}_y$, $\dot{\psi}$ about $\mathbf{e}_z$. Adding them:

$$\boldsymbol{\omega}_b = E(\boldsymbol{\theta}_b)\,\dot{\boldsymbol{\theta}}_b, \qquad E(\boldsymbol{\theta}_b) = \begin{bmatrix} c_\psi c_\theta & -s_\psi & 0 \\ s_\psi c_\theta & c_\psi & 0 \\ -s_\theta & 0 & 1 \end{bmatrix}, \qquad \det E = \cos\theta. \tag{6}$$

Its time derivative (needed in Steps 7 and 10):

$$\dot{E} = \begin{bmatrix} -s_\psi c_\theta\dot{\psi} - c_\psi s_\theta\dot{\theta} & -c_\psi\dot{\psi} & 0 \\ c_\psi c_\theta\dot{\psi} - s_\psi s_\theta\dot{\theta} & -s_\psi\dot{\psi} & 0 \\ -c_\theta\dot{\theta} & 0 & 0 \end{bmatrix}. \tag{7}$$

Watch out. **Gimbal lock.** At pitch $\theta = \pm 90°$, $\det E = 0$: roll and yaw turn about the same axis and one degree of freedom is lost. Near it, a small $\boldsymbol{\omega}_b$ needs huge Euler rates and the simulation becomes stiff or blows up. A standing or walking biped stays far from it; a falling one passes through it. `integrate()` warns when $|\theta| > 75°$.

Test — done when:
Numerically, $\log\big(R(\boldsymbol{\theta}+\epsilon\dot{\boldsymbol{\theta}})R(\boldsymbol{\theta})^\mathsf{T}\big)/\epsilon = E\dot{\boldsymbol{\theta}}$ (error $\sim 10^{-8}$). $\det E = 1,\ 0.5,\ 0.017$ at pitch $0°, 60°, 89°$. Finite difference of E along $\dot{\boldsymbol{\theta}}$ matches $\dot{E}$.

Step 4 | Jacobians

[done: `frame_jacobian`, `com_jacobian`]

Goal. For any point on any link, the 3×16 matrices mapping $\dot{\mathbf{q}}$ to its linear velocity and to the link's angular velocity.

Math. Everything comes from the rigid-body rule $\dot{\mathbf{x}} = \dot{\mathbf{o}} + \boldsymbol{\omega} \times (\mathbf{x} - \mathbf{o})$. For a point $\mathbf{x}$ on link i (use $\mathbf{x} = \mathbf{p}_i$ for the CoM, or a sole corner for contact):

$$\dot{\mathbf{x}} = J_v(\mathbf{q})\,\dot{\mathbf{q}}, \qquad \boldsymbol{\omega}_i = J_\omega(\mathbf{q})\,\dot{\mathbf{q}}, \qquad J_v, J_\omega \in \mathbb{R}^{3\times 16}. \tag{8}$$

Build them column by column. Each column is the motion caused by one entry of $\dot{\mathbf{q}}$ with all others zero.

Columns	J_v	J_ω
0–2: base position $\dot{\mathbf{p}}_b$	I_3	0
3–5: Euler rates $\dot{\boldsymbol{\theta}}_b$	$-[\mathbf{x} - \mathbf{p}_b]_\times\, E(\boldsymbol{\theta}_b)$	$E(\boldsymbol{\theta}_b)$
$6 + j$: revolute ancestor joint j	$\mathbf{a}_j \times (\mathbf{x} - \mathbf{p}_j)$	$\mathbf{a}_j$
$6 + j$: prismatic ancestor joint j	$\mathbf{a}_j$	0
$6 + j$: joint j not an ancestor of link i	0	0

The base rows are the notes' "floating-base columns": base rotation turns the whole robot about $\mathbf{p}_b$ with $\boldsymbol{\omega}_b = E\dot{\boldsymbol{\theta}}_b$. Stacked as one 6×16 matrix $[J_v; J_\omega]$ in the code.

CoM Jacobian: $J_{\text{CoM}} = \frac{1}{m}\sum_i m_i J_{v_i}$. **Contact Jacobian** for a point contact: $J_c = J_v$ at the contact point; for a flat foot fixed to the ground: $J_c = [J_{v_f}; J_{\omega_f}] \in \mathbb{R}^{6\times 16}$.

Test — done when:
Finite differences: column k of $J_v \approx (\mathbf{x}(\mathbf{q} + \epsilon\mathbf{e}_k) - \mathbf{x}(\mathbf{q}))/\epsilon$; column k of $J_\omega \approx \log(R_i(\mathbf{q} + \epsilon\mathbf{e}_k)R_i(\mathbf{q})^\mathsf{T})/\epsilon$. Error $< 10^{-6}$ at random poses (`step2_jacobian_check.py`). Hand check: standing straight, left knee only at $+1$ rad/s gives ankle velocity $(-0.4, 0, 0)$ and foot $\boldsymbol{\omega} = (0, 1, 0)$.

Step 5 | Mass matrix $M(\mathbf{q})$

[to do: `mass_matrix(q)`]

Goal. The 16×16 matrix with kinetic energy $T = \frac{1}{2}\dot{\mathbf{q}}^\mathsf{T} M(\mathbf{q})\dot{\mathbf{q}}$.

Math. Each link's kinetic energy is translation of its CoM plus rotation about its CoM:

$$T_i = \tfrac{1}{2}m_i\mathbf{v}_i^\mathsf{T}\mathbf{v}_i + \tfrac{1}{2}\boldsymbol{\omega}_i^\mathsf{T} I_i^W \boldsymbol{\omega}_i = \tfrac{1}{2}\dot{\mathbf{q}}^\mathsf{T}\left[m_i J_{v_i}^\mathsf{T} J_{v_i} + J_{\omega_i}^\mathsf{T} I_i^W J_{\omega_i}\right]\dot{\mathbf{q}} \quad \Longrightarrow \quad \boxed{M(\mathbf{q}) = \sum_i \left[m_i J_{v_i}^\mathsf{T} J_{v_i} + J_{\omega_i}^\mathsf{T} I_i^W J_{\omega_i}\right]} \tag{9}$$

where J_{v_i} is taken at the link's **center of mass** (`local_point = link.com`), not its frame origin.

Implement.

```
def mass_matrix(self, q):
    F = self.forward_kinematics(q)              # compute frames once
    M = np.zeros((self.nv, self.nv))
    for name, link in self.links.items():
        if link.mass == 0: continue
        J = self.frame_jacobian(q, name, link.com, F)
        Jv, Jw = J[0:3], J[3:6]
        R = F[name][0]
        IW = R @ link.inertia @ R.T             # world inertia about the CoM
        M += link.mass * Jv.T @ Jv + Jw.T @ IW @ Jw
    return M
```

Test — done when:
(a) $M = M^\mathsf{T}$ to 10^{-15}. (b) All eigenvalues > 0 (positive definite). (c) $M_{0:3,0:3} = 24.004\, I_3$ (total mass). (d) $\frac{1}{2}\dot{\mathbf{q}}^\mathsf{T} M\dot{\mathbf{q}} = \sum_i T_i$ computed link by link. (e) Hand check at the zero pose: the knee diagonal entry equals the inertia of shank + foot about the knee axis, $0.0405 + 3(0.2)^2 + 0.0035 + 1\,(0.05^2 + 0.45^2) = 0.3691$. Reference zero-pose diagonal: [24.004 (×3), 4.304, 4.130, 0.204, 2.058, 2.064, 0.369, 0.0085, 0.0028, …].

Step 6 | Gravity vector $G(\mathbf{q})$

[to do: `gravity_vector(q)`]

Goal. The generalized gravity force, written on the left-hand side of the equation of motion.

Math. The potential energy is $V(\mathbf{q}) = \sum_i m_i g\, z_i(\mathbf{q})$, where z_i is the height of link i's CoM. Since $\partial z_i/\partial \mathbf{q} = \mathbf{e}_z^\mathsf{T} J_{v_i}$:

$$\boxed{G(\mathbf{q}) = \frac{\partial V}{\partial \mathbf{q}} = \sum_i m_i g\, J_{v_i}^\mathsf{T}\mathbf{e}_z}, \qquad \mathbf{e}_z = [0, 0, 1]^\mathsf{T}. \tag{10}$$

The force gravity actually applies is $-G$; it moves to the left-hand side, hence the $+G$ there.

Implement. Same loop as Step 5: `G += link.mass * g * Jv.T @ ez`. Equivalently $G = mg\, J_{\text{CoM}}^\mathsf{T}\mathbf{e}_z$.

Test — done when:
(a) $G_{0:3} = (0,\ 0,\ 235.48)$ N $= mg\,\mathbf{e}_z$. (b) G matches a finite difference of $V(\mathbf{q})$ (error $\sim 10^{-8}$). (c) Zero pose: $G_{\text{pitch}} = -0.981$, every hip/knee/ankle-pitch entry $= -0.4905$ ($= 1\text{ kg} \cdot 9.81 \cdot 0.05$ m, because the foot CoM is 5 cm forward of the ankle); all others 0.

Step 7 | Coriolis and centrifugal term $C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$

[to do]

Goal. The velocity-dependent forces. Together with Step 6: $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = C\dot{\mathbf{q}} + G$.

There are two ways. Implement **A** first, because it is a direct transcription of the notes; then **B**, which is faster; then check that they agree.

Option A — Christoffel symbols (from M)

$$C_{ij}(\mathbf{q}, \dot{\mathbf{q}}) = \tfrac{1}{2} \sum_k \left[\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right] \dot{q}_k. \tag{11}$$

Obtain $\partial M / \partial q_k$ by central differences of `mass_matrix`: $(M(\mathbf{q} + \epsilon \mathbf{e}_k) - M(\mathbf{q} - \epsilon \mathbf{e}_k))/2\epsilon$, $\epsilon = 10^{-6}$. That is 32 mass-matrix evaluations per call: fine for testing, too slow for long simulations. (Where the formula comes from: Appendix A.)

Option B — Newton–Euler form (from Jacobians)

Apply Newton and Euler to every link and project with the Jacobians (Appendix A.3). This gives $C\dot{\mathbf{q}}$ directly, without differentiating M:

$$\boxed{C\dot{\mathbf{q}} = \sum_i \left[m_i J_{v_i}^{\mathsf{T}} \left(\dot{J}_{v_i} \dot{\mathbf{q}} \right) + J_{\omega_i}^{\mathsf{T}} \left(I_i^W \dot{J}_{\omega_i} \dot{\mathbf{q}} + \boldsymbol{\omega}_i \times I_i^W \boldsymbol{\omega}_i \right) \right]} \tag{12}$$

$\dot{J}\dot{\mathbf{q}}$ is the acceleration of the point when $\ddot{\mathbf{q}} = 0$. Simplest way to compute it: a central difference of the Jacobian *along the motion*,

$$\dot{J}\,\dot{\mathbf{q}} \approx \frac{J(\mathbf{q} + \epsilon\dot{\mathbf{q}}) - J(\mathbf{q} - \epsilon\dot{\mathbf{q}})}{2\epsilon}\,\dot{\mathbf{q}}, \qquad \epsilon = 10^{-7}. \tag{13}$$

Exact alternative: differentiate the table of Step 4 column by column, using $\dot{\mathbf{a}}_j = \boldsymbol{\omega}_{\text{child}(j)} \times \mathbf{a}_j$, $\frac{d}{dt}[\mathbf{a}_j \times (\mathbf{x} - \mathbf{p}_j)] = \dot{\mathbf{a}}_j \times (\mathbf{x} - \mathbf{p}_j) + \mathbf{a}_j \times (\dot{\mathbf{x}} - \dot{\mathbf{p}}_j)$, and $\dot{E}$ from Step 3 for the base columns.

```
def nonlinear_effects(self, q, qd):          # returns h = C qd + G
    F = self.forward_kinematics(q)
    h = self.gravity_vector(q)
    for name, link in self.links.items():
        if link.mass == 0: continue
        J  = self.frame_jacobian(q, name, link.com, F)
        Jd = (self.frame_jacobian(q + 1e-7*qd, name, link.com)
            - self.frame_jacobian(q - 1e-7*qd, name, link.com)) / 2e-7
        R  = F[name][0];  IW = R @ link.inertia @ R.T
        w  = J[3:6] @ qd
        h += link.mass * J[0:3].T @ (Jd[0:3] @ qd) \
           + J[3:6].T @ (IW @ (Jd[3:6] @ qd) + np.cross(w, IW @ w))
    return h
```

Test — done when:
(a) Options A and B give the same $C\dot{\mathbf{q}}$ at random $(\mathbf{q}, \dot{\mathbf{q}})$ (difference $\sim 10^{-9}$ relative to $|\mathbf{h}| \approx 240$). (b) With A's C: $\dot{M} - 2C$ is skew-symmetric, i.e. $\dot{M} - 2C + (\dot{M} - 2C)^{\mathsf{T}} = 0$, with $\dot{M} = \sum_k (\partial M / \partial q_k)\dot{q}_k$. (c) $C\dot{\mathbf{q}} = 0$ when $\dot{\mathbf{q}} = 0$.

Step 8 | Free-flight simulation (no contact)

[to do: `forward_dynamics`, `simulate`]

Goal. Simulate the robot falling freely: the first test of M, G and C together.

Math. Without contact the equation of motion is $M\ddot{\mathbf{q}} + \mathbf{h} = S^\mathsf{T}\boldsymbol{\tau}$, so

$$\ddot{\mathbf{q}} = M(\mathbf{q})^{-1}\big(S^\mathsf{T}\boldsymbol{\tau} - \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})\big), \qquad S^\mathsf{T}\boldsymbol{\tau} = [\,0_6;\ \boldsymbol{\tau}\,]. \tag{14}$$

Use `np.linalg.solve(M, rhs)`, not an explicit inverse. Then step forward in time. For Euler angles, $\dot{\mathbf{q}}$ is the true derivative, so an ordinary integrator works:

$$\dot{\mathbf{q}}_{k+1} = \dot{\mathbf{q}}_k + \Delta t\,\ddot{\mathbf{q}}_k, \qquad \mathbf{q}_{k+1} = \mathbf{q}_k + \Delta t\,\dot{\mathbf{q}}_{k+1} \qquad \text{(semi-implicit Euler, } \Delta t \le 1 \text{ ms)}. \tag{15}$$

For accuracy tests use RK4 on the state $[\mathbf{q}; \dot{\mathbf{q}}]$; for contact simulations (Step 9) semi-implicit Euler is the usual choice.

Test — done when:
(a) Free fall from rest, $\boldsymbol{\tau} = 0$: the CoM acceleration $J_{\text{CoM}}\ddot{\mathbf{q}} + \dot{J}_{\text{CoM}}\dot{\mathbf{q}}$ is exactly $(0, 0, -9.81)$, even with random joint velocities (error $\sim 10^{-10}$). (b) Total energy $E = \frac{1}{2}\dot{\mathbf{q}}^\mathsf{T} M\dot{\mathbf{q}} + V$ stays constant: RK4, $\Delta t = 2$ ms, 0.3 s, drift $\sim 10^{-10}$ J out of 149 J. If energy grows, C or G has a sign error. (c) Horizontal CoM velocity and total angular momentum about the CoM stay constant (no external horizontal force or moment).

Step 9 | Ground contact with springs (penalty model)

[to do]

Goal. The robot stands on the floor. Contact becomes an ordinary force, so Step 8 is reused unchanged.

Contact points. Use four sole corners per foot, in the foot frame: $x \in \{-0.05,\ 0.15\}$, $y \in \{-0.04,\ 0.04\}$, $z = -0.075$ (the bottom of the foot cylinder). Two points on the foot centerline (heel and toe only) cannot resist rolling, so ankle roll and lateral balance would fail.

Force law for each point with height $\phi = z$ and velocity $\dot{\mathbf{x}} = J_v\dot{\mathbf{q}}$:

$$f_n = \max\big(0,\ -k\,\phi - d\,\dot{\phi}\big) \quad \text{if } \phi < 0, \text{ else } 0 \qquad \text{(normal: spring + damper, never pulling)} \tag{16}$$

$$\mathbf{f}_t = -\mu f_n \frac{\dot{\mathbf{x}}_t}{\max(\|\dot{\mathbf{x}}_t\|,\ v_\epsilon)} \qquad \text{(friction: Coulomb, smoothed near zero speed)} \tag{17}$$

with $\dot{\mathbf{x}}_t$ the horizontal part of $\dot{\mathbf{x}}$. Each force enters the dynamics as a generalized force:

$$M\ddot{\mathbf{q}} + \mathbf{h} = S^\mathsf{T}\boldsymbol{\tau} + \sum_{\text{points } c} J_{v,c}^\mathsf{T}\,\mathbf{f}_c, \qquad \mathbf{f}_c = [\mathbf{f}_t;\ f_n]. \tag{18}$$

Starting values (tune them): standing on 8 points gives ≈ 29 N per point, so $k = 3 \times 10^4$ N/m gives ≈ 1 mm penetration; $d = 300$–600 N s/m; $\mu = 0.8$; $v_\epsilon = 10^{-3}$ m/s; $\Delta t = 0.5$–1 ms. A stiffer spring needs a smaller Δt.

Watch out. The smoothed friction lets a standing robot creep very slowly. That is acceptable at first. If it matters, use an anchored tangential spring at each sticking point, or move to Step 10.

Test — done when:
(a) Drop from 5 cm with all joints locked by stiff PD (Step 11): it bounces a little and settles. (b) At rest, the sum of all normal forces = 235.5 N and penetration ≈ 1 mm. (c) With k doubled and Δt halved, the motion barely changes.

Step 10 | Rigid contact (optional, more accurate)

[later]

Goal. Exact sticking contact: no springs, no penetration, no parameters to tune, at the cost of a linear solve and contact-mode logic.

Constraint. A sticking point stays put: $\mathbf{x}_c(\mathbf{q}) = \mathbf{x}_c^*$. Differentiating twice:

$$J_c\dot{\mathbf{q}} = 0, \qquad J_c\ddot{\mathbf{q}} + \dot{J}_c\dot{\mathbf{q}} = 0. \tag{19}$$

Solve for accelerations and contact forces together (one linear system per time step):

$$\begin{bmatrix} M & -J_c^\mathsf{T} \\ J_c & 0 \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}} \\ \mathbf{f}_c \end{bmatrix} = \begin{bmatrix} S^\mathsf{T}\boldsymbol{\tau} - \mathbf{h} \\ -\dot{J}_c\dot{\mathbf{q}} - 2\alpha J_c\dot{\mathbf{q}} - \alpha^2(\mathbf{x}_c - \mathbf{x}_c^*) \end{bmatrix}. \tag{20}$$

The last two terms (Baumgarte stabilization, e.g. $\alpha \approx 20\ \mathrm{s}^{-1}$) pull back the drift that numerical integration causes. Use $\dot{J}_c\dot{\mathbf{q}}$ as in Step 7. Rows of J_c must be independent: with four corner points the foot has 12 rows for 6 DoF, so use the 6×16 flat-foot J_c instead, or a least-squares solve.

After solving, check the result is physically possible: every normal force ≥ 0 (otherwise that contact lifts off), and $\|\mathbf{f}_t\| \leq \mu f_n$ (otherwise it slips). If not, change the active contact set and solve again. At touchdown, apply an impact first: $M(\dot{\mathbf{q}}^+ - \dot{\mathbf{q}}^-) = J_c^\mathsf{T}\boldsymbol{\Lambda}$, $J_c\dot{\mathbf{q}}^+ = 0$.

Test — done when:
Standing still: $\mathbf{f}_c$ sums to $(0, 0, 235.5)$ N and $J_c\dot{\mathbf{q}}$ stays ≈ 0. Same standing pose as Step 9: the contact forces agree to within a few percent.

Step 11 | First controller: standing

[later]

Goal. Hold a slightly crouched standing pose.

Joint PD: $\boldsymbol{\tau} = K_p(\mathbf{q}_j^{\mathrm{des}} - \mathbf{q}_j) - K_d\dot{\mathbf{q}}_j$, with e.g. $K_p = 300$–500 N m/rad, $K_d = 10$–30 N m s/rad, clipped to the URDF effort limit (150 N m). Start from a slight crouch (hip -0.3, knee 0.6, ankle -0.3 rad) on penalty contact.

Test — done when:
The robot settles and stays up for 10 s with zero commanded motion; the CoM stays above the support polygon (the rectangle spanned by the sole corners).

Appendix A | Where the equations come from

A.1 Euler–Lagrange equation

With $L = T - V$ and generalized forces Q, Lagrange's equation is $\frac{d}{dt}\big(\partial L/\partial\dot{\mathbf{q}}\big) - \partial L/\partial\mathbf{q} = Q$. Since V does not depend on $\dot{\mathbf{q}}$ and $T = \frac{1}{2}\dot{\mathbf{q}}^\mathsf{T}M\dot{\mathbf{q}}$:

$$\frac{\partial T}{\partial\dot{\mathbf{q}}} = M\dot{\mathbf{q}}, \qquad \frac{d}{dt}\frac{\partial T}{\partial\dot{\mathbf{q}}} = M\ddot{\mathbf{q}} + \dot{M}\dot{\mathbf{q}}, \qquad \frac{\partial T}{\partial q_i} = \tfrac{1}{2}\dot{\mathbf{q}}^\mathsf{T}\frac{\partial M}{\partial q_i}\dot{\mathbf{q}}. \tag{21}$$

So row i is

$$\sum_j M_{ij}\ddot{q}_j + \underbrace{\sum_{j,k}\frac{\partial M_{ij}}{\partial q_k}\dot{q}_k\dot{q}_j}_{(\dot{M}\dot{\mathbf{q}})_i} - \tfrac{1}{2}\sum_{j,k}\frac{\partial M_{jk}}{\partial q_i}\dot{q}_j\dot{q}_k + G_i = Q_i. \tag{22}$$

A.2 The step to the Christoffel form (missing in the original notes)

In the middle sum, j and k are both summed over, so swapping their names changes nothing. Write the sum as half of itself plus half of the swapped version:

$$\sum_{j,k} \frac{\partial M_{ij}}{\partial q_k}\dot{q}_k\dot{q}_j = \tfrac{1}{2}\sum_{j,k}\Big[\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j}\Big]\dot{q}_j\dot{q}_k. \tag{23}$$

Combine with the last sum, and factor out one $\dot{q}_j$:

$$\sum_j \underbrace{\tfrac{1}{2}\sum_k \Big[\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i}\Big]\dot{q}_k}_{C_{ij}}\ \dot{q}_j = (C\dot{\mathbf{q}})_i. \tag{24}$$

Hence $M\ddot{\mathbf{q}} + C\dot{\mathbf{q}} + G = Q$. This particular C makes $\dot{M} - 2C$ skew-symmetric (energy balance $\dot{T} + \dot{V} = \dot{\mathbf{q}}^{\mathsf{T}}Q$), which is test (b) of Step 7.

A.3 Newton–Euler form (Option B of Step 7)

For each link, Newton: $m_i\dot{\mathbf{v}}_i = \mathbf{F}_i$; Euler about the CoM in world axes: $I_i^W\dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times I_i^W\boldsymbol{\omega}_i = \mathbf{N}_i$ (the cross term comes from I_i^W rotating with the link). Multiply each by its Jacobian transpose and add up over links; internal joint forces cancel because they do no work. Substituting $\dot{\mathbf{v}}_i = J_{v_i}\ddot{\mathbf{q}} + \dot{J}_{v_i}\dot{\mathbf{q}}$ and $\dot{\boldsymbol{\omega}}_i = J_{\omega_i}\ddot{\mathbf{q}} + \dot{J}_{\omega_i}\dot{\mathbf{q}}$ gives $M\ddot{\mathbf{q}}$ (the same M as Step 5) plus the boxed $C\dot{\mathbf{q}}$ of Step 7. $C\dot{\mathbf{q}}$ is identical to Option A, although the matrix C itself can be written in more than one way.

A.4 Generalized forces by virtual work

A generalized force Q is defined by: the power delivered equals $\dot{\mathbf{q}}^{\mathsf{T}}Q$.

- **Joint torque** τ_j acts equally and oppositely on parent and child, delivering power $\tau_j\dot{q}_j$. It appears only in row $6+j$, so $Q = S^{\mathsf{T}}\boldsymbol{\tau}$; the 6 base rows get nothing (the base is unactuated).
- **Contact force** $\mathbf{f}_c$ at a point moving with $J_c\dot{\mathbf{q}}$ delivers power $\mathbf{f}_c^{\mathsf{T}}J_c\dot{\mathbf{q}}$, so $Q = J_c^{\mathsf{T}}\mathbf{f}_c$.
- **Gravity** delivers power $-\dot{V} = -\dot{\mathbf{q}}^{\mathsf{T}}G$, so $Q = -G$; moved to the left-hand side it becomes $+G$.

Result: $M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + G(\mathbf{q}) = S^{\mathsf{T}}\boldsymbol{\tau} + J_c^{\mathsf{T}}\mathbf{f}_c$.

Appendix B | Implementation checklist

Step	Function	Must pass	
1	`RobotModel(urdf)`	10 joints, $n_q = 16$, 24.004 kg	☐
2	`forward_kinematics(q)`	zero-pose positions, CoM $z = 0.5625$	☐
3	`euler_rate_matrix(rpy)`	$\boldsymbol{\omega} = E\dot{\boldsymbol{\theta}}$ vs finite difference	☐
4	`frame_jacobian`, `com_jacobian`	finite-difference check $< 10^{-6}$	☐
5	`mass_matrix(q)`	symmetric, positive definite, $24.004\,I_3$ block, knee 0.3691	☐
6	`gravity_vector(q)`	$G_z = 235.48$; matches $\partial V/\partial\mathbf{q}$	☐
7	`coriolis_matrix`, `nonlinear_effects`	A = B; $\dot{M} - 2C$ skew	☐
8	`forward_dynamics`, `simulate`	CoM falls at -9.81; energy constant	☐
9	`contact_forces(q, qd)`	stands; $\sum f_n = 235.5$ N	☐
10	`constrained_dynamics`	$f_n \geq 0$, $J_c\dot{\mathbf{q}} \approx 0$	☐
11	`pd_controller`	stands for 10 s	☐